SAPIENZA
UNIVERSITÀ DI ROMA

Implementation of the Einstein sum

Department of Computer Science
Applied Computer Science and artificial intelligence

Domenico Menale
ID number 2049488

Advisor
Prof. Maria De Marsico

Co-Advisor
Dr. Diego Bellani

Academic Year 2025/2026

Thesis not yet defended

Implementation of the Einstein sum
Bachelor degree. Sapienza University of Rome

© 2026 Domenico Menale. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: menale.2049488@studenti.uniroma1.it

Imagination is more important than knowledge
Albert Einstein

Abstract

This work explores different computational implementations of the Einstein summation convention, a powerful mathematical notation for expressing tensor contractions used across scientific computing, physics, engineering, and modern machine learning. Particularly in neural networks, einsum notation expresses operations such as batch matrix multiplication, multi-head attention transformations, and tensor reshaping—operations that would otherwise require cumbersome loops or explicit memory reorganization. While the notation itself provides abstraction for multi-dimensional array operations, the primary focus of this thesis is the practical challenge of implementing it: translating abstract einsum specifications into executable code. We present four distinct implementation strategies: generalized odometer iteration, just-in-time code generation, tensor permutation and flattening, and BLAS-accelerated dispatch. Each has different trade-offs between flexibility, compilation overhead, and runtime performance. We analyze these approaches by benchmarking against NumPy's optimized einsum and validate their correctness across diverse tensor dimensions and contraction patterns.

Contents

1 Introduction to Einstein summation and tensors **1**
1.1 Tensors memory layout and access patterns 1
1.2 Einsum in the Architecture of Neural Networks 3
1.2.1 Linear layers and batch processing 3
1.2.2 Broader applications 3
1.3 From Mathematical Notation to algorithm 4
1.3.1 The Compilation Pipeline 4
1.3.2 Execution Strategies and Algorithmic Diversity 4

2 Implementations of the Einstein sum **7**
2.1 Generalized Tensor Contraction via Odometer Iteration 7
2.1.1 The Technical Architecture 7
2.1.2 Execution and Performance Characteristics 8
2.2 Just-In-Time (JIT) Code Generation and AOT Specialization 8
2.2.1 Optimization Mechanisms 8
2.2.2 Summary of the JIT Engine 9
2.3 Canonical Contraction through Permutation and Flattening 9
2.3.1 The Transformation Pipeline 9
2.4 BLAS-Accelerated Pattern Recognition and Greedy Dispatch 11
2.4.1 The Pattern Hierarchy 11
2.4.2 Intelligent Dispatch Logic 12
2.4.3 Future work: stride-based optimization 12
2.4.4 Summary of the BLAS Engine 12

3 Performance analysis **13**
3.1 Experimental setup 13
3.1.1 Hardware and software configuration 13
3.1.2 Measurement methodology 13
3.2 Test case selection and rationale 14
3.3 Statistical modeling of execution times 14
3.3.1 The Lognormal Distribution 14
3.4 Benchmark Results and Comparative Analysis 16
3.4.1 Contraction on different tensor sizes 16
3.4.2 Summary of Implementation Trade-offs 18
3.5 Higher-Dimensional Tensor Comparisons 18

4 Conclusions **21**
4.1 Summary 21
4.2 Future works 22

Chapter 1

Introduction to Einstein summation and tensors

For our work we need to first define what is a tensor and what is the Einstein summation convention. Tensors are a widely used representations of multidimensional data in scientific and engineering applications, as discussed in [5]. In the physical world—specifically in architectural engineering and structural analysis—tensors represent physical quantities that are independent of any specific coordinate system. For example, the stress tensor σ_{ij} describes the internal forces within a solid body. To calculate the interaction between these multi-dimensional objects, we can use a robust notation: the **Einstein Summation Convention**.

The summation convention was introduced by Einstein in 1916 [2]. It implies that whenever an index in a term is repeated, once as a superindex and once as a subindex, a summation over that index is understood. With these conventions, matrix vector multiplication is written as defined in [9]

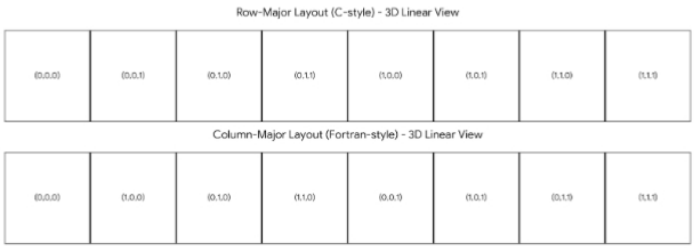
Consider the transformation of a vector v by a matrix M:

$$y_i = \sum_{j=1}^{n} M_{ij} v_j \quad \longrightarrow \quad y_i = M_{ij} v_j \tag{1.1}$$

This notation formalizes the contraction process: a linear mapping that reduces the rank of a tensor by summing over one or more indices.

1.1 Tensors memory layout and access patterns

In the implementations that we will see in the next chapter, tensors are stored in contiguous memory buffers. The way these multi-dimensional arrays are laid out in memory is crucial for performance. The two most common layouts are row-major (C-style) and column-major (Fortran-style). In a row-major layout, the last index changes fastest, while in column-major, the first index changes fastest. For example, a 3D tensor T_{ijk} with shape $(2, 2, 2)$ would be stored as:



The choice of layout affects the memory access patterns during tensor contractions. Accessing elements that are stored contiguously in memory leads to better cache performance and faster execution. For instance, in a row-major layout, iterating over the last index k of T_{ijk} will be more efficient than iterating over the first index i. Furthermore we can define the **stride** of a tensor as the number of elements to skip in memory to move from one index to the next. For a tensor with shape (d_0, d_1, d_2) in row-major order, the strides would be:

$$\text{stride}_0 = d_1 \cdot d_2, \quad \text{stride}_1 = d_2, \quad \text{stride}_2 = 1 \tag{1.2}$$

and the offset as

$$\text{Offset} = i \cdot \text{stride}_0 + j \cdot \text{stride}_1 + k \cdot \text{stride}_2 \tag{1.3}$$

Understanding these concepts allows us to implement efficient tensor operations, as they directly impact the performance of algorithms that rely on tensor contractions, such as those expressed in Einstein summation notation.

A critical factor in the performance of tensor operations is the computer's memory hierarchy. As described in [7], modern CPUs employ a stratified memory system: processors execute instructions on data stored in registers, but must regularly fetch operands from main memory, which is orders of magnitude slower. Between the CPU and main memory sit intermediate caches (L1, L2, L3) that store recently-accessed data. A cache hit—when the CPU finds needed data in the cache—incurs minimal latency, while a cache miss—requiring a fetch from main memory—incurs substantial delay. The total execution time of an algorithm is therefore determined not just by the number of floating-point operations, but critically by how well the data access pattern exploits the cache hierarchy. Tensors that fit entirely within faster cache levels execute much faster than those stored in slower memory. This is why choosing appropriate memory layouts and access patterns, as discussed above, is essential for achieving high performance.

1.2 Einsum in the Architecture of Neural Networks

In the modern computational landscape, tensors are the primary data structure for Deep Learning.

1.2.1 Linear layers and batch processing

In modern deep learning architectures, a Linear Layer serves as an affine transformation that projects an input tensor into a new feature space via a learned weight matrix. This operation is fundamentally a tensor contraction, which can be concisely expressed using the Einstein summation convention. Consider a weight matrix W with dimensions (I, O), where I is the number of input features and O is the number of output features. If we process a single vector x of size I, the operation is a standard matrix-vector product. However, with a batch size B, our input becomes a 2D tensor $X \in \mathbb{R}^{B \times I}$.

Using the `einsum` notation, the operation is expressed as:

$$Y_{bo} = X_{bi} W_{io} \tag{1.4}$$

This specific notation provides several computational advantages:

Feature mapping: The index i is the "contraction dimension." It represents the internal connection where each input feature is weighted and summed to contribute to an output feature.

Batch preservation: The index b is a "broadcast index." It remains untouched by the summation, meaning the same weights W are applied identically to every sample in the batch.

Parallelism: By representing the batch as a tensor dimension, the underlying `einsum` engine can dispatch the entire operation as a single `GEMM` (General Matrix Multiply) call.

Without the batch dimension, the hardware would have to launch B separate, smaller kernels, leading to significant overhead and under-utilization of the high-speed memory bus. Thus, the Einstein notation does not just describe the operations, it defines the memory access pattern.

1.2.2 Broader applications

The utility of the Einstein summation convention extends far beyond the optimization of neural networks; it can describe physical systems where multi-axial forces interact.

Continuum Mechanics and Structural Elasticity

In structural engineering, the linear elastic response of a material—such as a reinforced concrete boundary—is governed by the generalized **Hooke's Law**. This relationship defines the second-order stress tensor σ_{ij} as the result of a contraction between a fourth-order elasticity (stiffness) tensor $\mathcal{C}_{ijkl}$ and a second-order strain tensor ε_{kl} [4]:

$$\sigma_{ij} = \mathcal{C}_{ijkl}\varepsilon_{kl} \tag{1.5}$$

This operation involves a double contraction over indices k and l, effectively reducing a system of 81 potential coefficients in the elasticity tensor to a 9-component stress representation. Within the framework developed in this work, this is concisely expressed as `einsum('ijkl,kl->ij')`. Such a formulation can be used to simulate how external loads or thermal expansion distribute internal forces across complex architectural boundaries [3].

1.3 From Mathematical Notation to algorithm

The translation of a symbolic string like `"ij,jk->ik"` into machine code is not a trivial task. The implementation must bridge the gap between high-level logic and Low-Level Stride Arithmetic.

1.3.1 The Compilation Pipeline

When a user calls an `einsum` function, the system undergoes several stages of translation:

Parsing The string is analyzed to create bitmasks representing index occupancy.

Contraction Mapping The engine identifies which indices are shared (summation) and which are unique (free).

Memory Stride Calculation To access data in a flat memory buffer, the engine calculates the *stride*—the number of elements to skip to reach the next index in a given dimension.

1.3.2 Execution Strategies and Algorithmic Diversity

Since the Einstein notation is so expressive, we present different implementations that fit different use cases, strarting with a general unoptimized algorithm and moving towards better performing options.

Generalized Odometer Interpretation: The most flexible approach is the Odometer-based iteration. This strategy treats the N-dimensional space as a single flat counter. It identifies every unique index in the notation (both free and summed) and builds a virtual “odometer”, an instrument used to measure distances traveled by cars, where each gear corresponds to a dimension size. Every time the odometer ticks the counter advances by one in the lowest dimension, and when it overflows, it resets to zero and advances the next gear or, in our case, the next dimension in the tensor. In computational terms, this allows us to simulate a nested loop structure of arbitrary depth N without requiring hard-coded loops. By maintaining a coordinate vector that "ticks" forward,

the system can map every multidimensional index $(i_0, i_1, \ldots, i_n)$ to a linear memory offset using the formula:

$$\text{Offset} = \sum_{d=0}^{n-1} (idx_d \times \text{stride}_d) \tag{1.6}$$

While this method is capable of handling arbitrary rank-N tensors, where the rank is the number of dimensions of the tensor, without pre-compilation, it is limited by the overhead of coordinate-to-offset calculations performed inside the innermost loop. It serves as a study for the genuine implementation of the algorithm that is capable of handling any tensor.

JIT-Code Generation and Specialization: Another possible strategy is **Just-In-Time (JIT)** compilation. Instead of calling a library, the engine generates a raw C or C++ source file containing the exact nested loops required for the specific tensor shapes at hand.

By hardcoding the loop bounds and strides, the system allows the compiler to perform aggressive optimizations like SIMD vectorization and Loop Unrolling. This eliminates the "index lookup" overhead entirely, creating a zero-overhead kernel specialized for a single mathematical task.

Canonical Permutation and Flattening: By using a **Permute-Flatten** strategy we can treat the operation as a **matrix multiplication**. This involves reordering the axes of the input tensors so that the summation indices are grouped contiguously.

Once the data is rearranged, the multidimensional tensor is "flattened" into a 2D matrix view. This allows the system to treat a 4D or 6D contraction as a simple 2D `GEMM` operation, drastically reducing the complexity from $O(d^m)$ to $O(M \cdot K \cdot N)$, where M, K, N are the products of the respective dimension groups.

BLAS Pattern Dispatching: When treating 1D or 2D patterns (such as Dot, GER, or GEMV), the best solution is to route the data to **Basic Linear Algebra Subprograms (BLAS)**. These are hardware-vendor-tuned kernels (like Intel MKL or Apple Accelerate) that use cache-blocking and tiling techniques.

By utilizing BLAS, we ensure that standard operations achieve the peak theoretical FLOPS (Floating Point Operations Per Second) of the host CPU or GPU.

Chapter 2

Implementations of the Einstein sum

This chapter presents the different implementations of the Einstein sum.

2.1 Generalized Tensor Contraction via Odometer Iteration

This first implementation provides a generalized approach to tensor contraction by treating the dimensions of the tensors as units of measure for an **Odometer**. This object allows the system to systematically iterate through all possible index combinations required by the Einstein summation notation. This method is built upon the principle of explicit summation, where every calculation is performed by manually traversing each coordinate of the tensors involved.

While this approach offers high clarity for educational purposes and flexibility across tensor ranks, its computational complexity grows exponentially. For a tensor of rank n and dimension d, the total iterations required is d^n, making it a robust "one-size-fits-all" structure primarily suitable for small datasets.

2.1.1 The Technical Architecture

The implementation relies on three core components to manage the complexity of multi-dimensional spaces: Lexicographic Stepping, Bitwise Labeling, and Linear Offsetting.

Lexicographic Stepping (next_indices): The core of the iteration logic is the `next_indices` function. This simulates a multi-dimensional counter that increments in row-major order.

- It identifies the least-significant dimension (the fastest-changing index).
- It increments the index until the maximum bound is reached, at which point it "carries" the increment to the next more-significant dimension.

This logic allows a single flat loop to simulate an arbitrary number of nested loops, enabling the code to handle any tensor rank dynamically.

Bitwise Labeling (IndexBitmap): To manage Einstein notation labels (e.g., ‘ i ’, ‘ j ’, ‘ k ’), the system utilizes an `IndexBitmap`. This converts lowercase letters into a 26-bit integer.

$$\text{Bitmap} = \sum \text{bit}(c - \text{'a'})$$

This allows for nearly instantaneous membership tests. Finding common indices between two tensors is reduced to a simple bitwise `AND` operation, eliminating expensive string comparisons during the inner loops of the summation.

Linear Offset Calculation: Data is stored in a contiguous memory buffer following a row-major model. The `compute_offset` function translates a multi-dimensional coordinate into a single memory address:

$$\text{Offset} = \sum_{i=0}^{n-1} \left(\text{index}_i \cdot \prod_{j=i+1}^{n-1} \text{shape}_j \right)$$

While this provides $O(n)$ access time, the dynamic recalculation of these offsets during each iteration step accounts for the primary logic overhead of this approach.

2.1.2 Execution and Performance Characteristics

In the final execution stage, the `einsum` function creates a “combined iteration shape” encompassing every unique index in the notation. The odometer iterates across this high-dimensional space, multiplying values and accumulating them into the result.

2.2 Just-In-Time (JIT) Code Generation and AOT Specialization

This implementation represents a Meta-Programming approach to tensor operations. Instead of relying on a generalized function that interprets notation at runtime, this system acts as a domain-specific compiler. It writes custom-tailored C source code for specific tensor shapes and notations, which is then compiled into a highly optimized binary kernel.

The primary shift here is the move from runtime interpretation to Ahead-of-Time (AOT) Specialization.

2.2.1 Optimization Mechanisms

The generator utilizes three primary optimization strategies: Literal Stride Injection, Hierarchical Loop Ordering, and Compiler-Assisted Vectorization.

Literal Stride Injection: In generalized functions, strides are looked up in arrays at runtime. This generator calculates these strides once and prints them as literal constants.

For an index “ij” with shape $[7, 3]$, the generator outputs the direct arithmetic `(i * 3) + j`.

Hierarchical Loop Ordering: The generator enforces a strict hierarchy to maximize data locality:

Outer Shell: Iterates over the Free Indices (those present in the output), ensuring a contiguous and organized traversal of the destination memory.

Inner Core: Iterates over Summation Indices.

This structure maximizes Temporal Locality by keeping the accumulation sum for a specific output element within a high-speed CPU register for the entire duration of the summation loop.

Compiler-Driven Vectorization: By outputting high-level C code rather than raw assembly, the implementation leverages the full power of modern optimizers.

Because loop bounds (e.g., `i < 7`) are known at compile time, the compiler can perform aggressive Loop Unrolling and automatically inject SIMD (Single Instruction, Multiple Data) instructions to process multiple floating-point operations in parallel.

2.2.2 Summary of the JIT Engine

This approach treats Einstein notation as a high-level language. By moving the computational burden from the runtime environment to the Compilation Phase, it reduces logic overhead and memory latency, making it adapt for production environments where tensor shapes are known in advance.

2.3 Canonical Contraction through Permutation and Flattening

This implementation introduces a transformation designed to bridge the gap between abstract tensor notation and linear algebra. Rather than iterating through a high-dimensional space with a generalized odometer, this approach transforms any arbitrary N-dimensional Einstein summation into a standard 2D matrix multiplication ($C = A \times B$).

By reducing the problem to a canonical matrix form, the system can leverage optimized hardware routines while maintaining the flexibility to handle complex tensor geometries.

2.3.1 The Transformation Pipeline

The core logic follows a three-stage pipeline: permute, flatten, and multiply. This strategy is designed to reduce any arbitrary N-dimensional contraction into a single, **GEMM** (general matrix multiplication) call.

Tensor Permutation and Grouping: To treat a tensor as a matrix, the memory layout must be rearranged so that relevant dimensions are contiguous. The function `matrix_permute` reorders the axes of the source tensors based on their roles in the Einstein notation:

Tensor A: Indices are grouped into [Free Indices, Summation Indices].

Tensor B: Indices are grouped into [Summation Indices, Free Indices].

This grouping ensures that the "right side" of Tensor A (the dimensions being summed over) aligns perfectly with the "left side" of Tensor B in linear memory.

Formal Proof via Matricization: The validity of this reduction follows from the theory of tensor unfolding or matricization as illustrated in [6]. Matricization is a bijective mapping that rearranges the elements of a tensor into a matrix without changing their values.

Proof. Consider the generalized tensor contraction

$$\mathcal{C}_{IJ} = \sum_K \mathcal{A}_{IK}\mathcal{B}_{KJ},$$

where I, J, and K denote index sets. The indices in K are contracted, while I and J remain as free indices.

We construct a bijective mapping ϕ that flattens each multi-index into a single integer index. In particular,

$$i = \phi(I), \qquad k = \phi(K), \qquad j = \phi(J).$$

Let

$$M = \prod \text{shape}(I), \qquad L = \prod \text{shape}(K), \qquad N = \prod \text{shape}(J).$$

Using this mapping, we define the matricizations

$$A \in \mathbb{R}^{M\times L}, \qquad B \in \mathbb{R}^{L\times N},$$

such that

$$A_{\phi(I),\phi(K)} = \mathcal{A}_{IK}, \qquad B_{\phi(K),\phi(J)} = \mathcal{B}_{KJ}.$$

Substituting these definitions into the contraction yields

$$C_{ij} = \sum_{k=1}^{L} A_{ik}B_{kj},$$

which is exactly the definition of matrix multiplication:

$$C = AB.$$

Since the mapping ϕ is bijective (it only reorders tensor elements according to the tensor's strides), no information is lost and the algebraic structure of the operation is preserved. Therefore, the original tensor contraction is equivalent to a matrix multiplication after matricization [8]. □

Flattening into 2D Matrices: Once dimensions are grouped, the implementation calculates the product of the shapes in each group to determine the effective number of rows and columns. Let I be the set of free indices in A, and K be the set of shared summation indices. Tensor A is treated as a matrix of size $M \times L$, where:

$$M = \prod_{i \in I} \text{shape}(i) \quad \text{and} \quad L = \prod_{k \in K} \text{shape}(k) \tag{2.1}$$

Once the permutation is complete, we pass the resulting memory pointer to the BLAS routine, effectively treating the multidimensional data as a 2D array.

Multiplication and Reshaping: With the tensors transformed, the code invokes a BLAS-compliant multiplication to compute the product $C = AB$ The result initially exists in a canonical order where all free indices from A appear before those from B. A final `matrix_permute` call is performed on the intermediate result to match the specific output notation requested by the user.

2.4 BLAS-Accelerated Pattern Recognition and Greedy Dispatch

This implementation utilizes a greedy pattern matching strategy to map Einstein notation to the highly optimized Basic Linear Algebra Subprograms (**BLAS**) library. Instead of treating every contraction as a generic tensor operation, this system attempts to identify standard linear algebra operations that can be executed using hardware-tuned Level-1, Level-2, or Level-3 BLAS routines.

2.4.1 The Pattern Hierarchy

The implementation utilizes a hierarchy of functions, prioritizing specialized kernels over more general ones to minimize computational overhead. A critical feature of this approach is that pattern matching works hierarchically: the dispatch logic first identifies the most specific applicable pattern, even when that pattern is nested within a larger dimensional structure. For example, a batched matrix multiplication like `"bij,bjk->bik"` is recognized as a GEMM operation (the core `"ij,jk->ik"` pattern) repeated across the batch dimension b. This allows the system to dispatch batched operations directly to optimized BLAS routines that handle them as a single composite operation, rather than treating them as generic rank-3 tensor contractions.

Level-1 Operations (Vector-Vector): DOT: Detects patterns like `"i,i->"` to perform a scalar dot product.

AXPY: Recognizes `"i,i->i"` for constant-multiplication and vector addition ($y := \alpha x + y$).

Level-2 Operations (Matrix-Vector): GER: Identifies `"i,j->ij"` to perform a rank-1 outer product.

GEMV: Matches `"ij,j->i"` or `"i,ij->j"` to execute matrix-vector multiplication, supporting both standard and transposed variants.

Level-3 Operations (Matrix-Matrix): GEMM: The most critical kernel, matching `"ij,jk->ik"`.

2.4.2 Intelligent Dispatch Logic

The system analyzes the input notation using bitwise masks and dimension counts to fill a `BLASAnalysis` metadata structure. This structure guides the main dispatch function, which routes the matrices to specific case handlers.

Pattern	BLAS Level	Mathematical Operation
DOT	Level 1	Scalar Product: $s = \sum x_i y_i$
AXPY	Level 1	Vector Accumulation: $y = \alpha x + y$
GER	Level 2	Outer Product: $A = xy^T$
GEMV	Level 2	Matrix-Vector: $y = Ax$
GEMM	Level 3	Matrix-Matrix: $C = AB$

Table 2.1. The greedy dispatch hierarchy used for BLAS acceleration.

2.4.3 Future work: stride-based optimization

This implementation serves as the basis for stride-based patterns. While the current version handles unit strides, standard BLAS functions accept an `incx` parameter.

In future iterations, this would allow the system to handle complex patterns like `"ii->"` (Matrix Trace) or `"xii,xii->"` (Diagonal Dot Product) without moving data. By calculating a custom stride based on the tensor shape, the engine could traverse diagonal elements as if they were a contiguous vector, further reducing the need for temporary memory allocations or tensor permutations.

2.4.4 Summary of the BLAS Engine

This approach allows us to leverage the BLAS library. By identifying the most specific "building block" for a given operation, we ensure that simple operations remain fast while complex matrix multiplications still take advantage of the optimized BLAS routines.

Chapter 3

Performance analysis

In this chapter we analyze the performance characteristics of the four Einstein summation implementations developed in Chapter 2. Our goal is to quantify the **practical trade-offs** between computational overhead and execution time across different problem scales. We benchmark against NumPy's industry-standard `einsum` implementation to contextualize our results. It is important to mention that Numpy uses a sophisticated engine that combines high specialized BLAS kernels with a fallback to specialized iterators.

3.1 Experimental setup

3.1.1 Hardware and software configuration

All benchmarks were executed on a machine with the following specifications:

CPU : Intel Core i7-9700K @ 3.60GHz

RAM : 32GB DDR4 @ 2666 MHz

OS : Ubuntu 20.04 LTS with Linux kernel 5.15

Compiler : GCC 9.3.0 with `-O3` optimization flags

BLAS Library : OpenBLAS 0.3.13 with 8 threads

3.1.2 Measurement methodology

We implemented the benchmarks in C, using the `clock_gettime(CLOCK_MONOTONIC)` function for timing. To minimize statistical noise, each benchmark case was executed 500 times. We use Python's NumPy library (version 1.21.0) as a reference point. While direct C-to-Python comparisons are inherently imperfect due to Python's interpretive overhead, NumPy's `einsum` function is implemented in optimized C with BLAS backend support, making it a fair baseline for the algorithmic choices (though not the language overhead).

3.2 Test case selection and rationale

The performance analysis focuses on three representative tensor contraction patterns, each chosen to expose different algorithmic characteristics. Three different tensor sizes are chosen to observe how cache interactions affect performance at different scales (data fitting in L1 cache, L2 cache, and main memory), validating the concepts introduced in Section 1.1.

Matrix multiplication (`ij,jk->ik`) This test case was chosen because it is the simplest and most common operation that can be expressed in Einstein notation. We tested three sizes to see how implementations behave at different scales:

(10, 10) Data fits in fast L1 cache.

(50, 50) Data partially in L2 cache.

(100, 100) Data too large for cache.

3D tensor contraction (`abc,cd->abd`) This operation represents a higher-rank contraction that does not trivially map to standard BLAS operations. It tests whether our implementations correctly handle rank-3 tensors and whether the Permute-Flatten strategy successfully reduces it to matrix multiplication.

3.3 Statistical modeling of execution times

Raw execution time measurements are inherently noisy due to operating system scheduling, cache behavior, and other system-level variability. To extract meaningful insights, we model execution times using a **lognormal distribution**, which is well-suited for positive-valued measurements with right-skewed distributions.

3.3.1 The Lognormal Distribution

The lognormal distribution is characterized by three key parameters: the location parameter μ (mean of the log-transformed data), the scale parameter σ (standard deviation of the log-transformed data), and the shift parameter τ (minimum threshold). Unlike the normal distribution, which ranges from $-\infty$ to $+\infty$, the lognormal distribution is strictly positive for $t > \tau$, making it ideal for performance measurements.

The key insight is that while a normal distribution can produce negative values, the lognormal distribution enforces strict positivity by design. This is critical for execution time analysis, where negative measurements are physically impossible. The right-skewed shape of the lognormal distribution accurately reflects how execution times cluster around a minimum value with occasional larger measurements due to system perturbations, cache misses, and scheduling delays.

For each implementation and tensor configuration, we collected 500 execution time samples. The log-transformed timings (i.e., $\ln(t_i)$ for each measurement t_i) approximate a normal distribution, validating the use of the lognormal model.

A normal distribution can produce negative values, making it inappropriate for strictly positive data like execution times. The lognormal distribution naturally

enforces positivity by construction. By modeling the logarithm of execution times, we can use well-understood statistical tools while respecting the physical constraints of the data.

Given n execution time samples $t_1, t_2, \ldots, t_n$, the parameters are computed as:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} \ln(t_i - \tau), \quad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\ln(t_i - \tau) - \mu)^2} \tag{3.1}$$

where μ is the mean of the shifted log-transformed execution times, σ is the standard deviation, and τ is the location parameter representing the minimum execution time (the inherent constant overhead that cannot be avoided regardless of computational complexity). The standard lognormal distribution with location shift ensures that all times satisfy $t \geq \tau$, capturing the fact that even a zero-cost operation requires some baseline CPU cycles. Estimating τ can be performed using order statistics or maximum likelihood methods; in practice, τ is typically slightly below the minimum observed execution time.

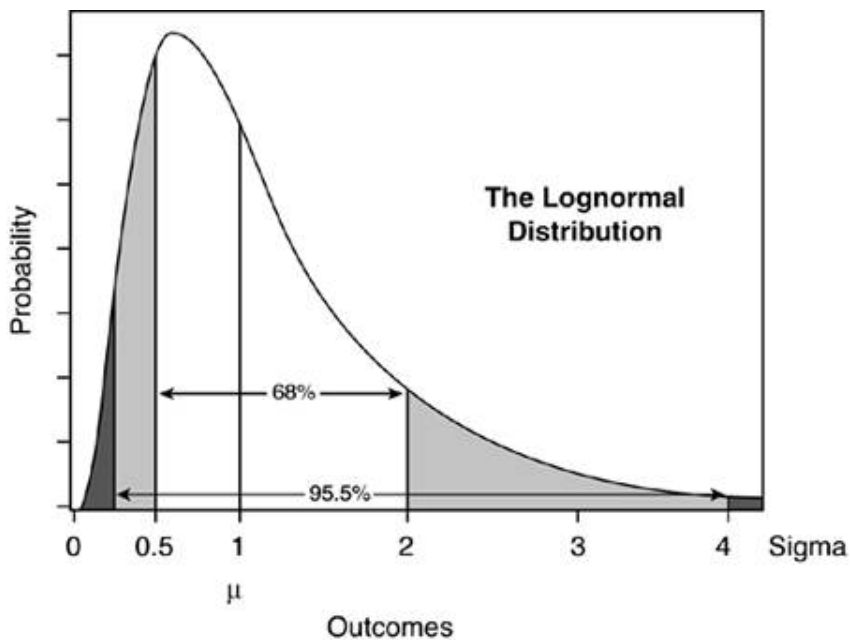


Figure 3.1. The shifted lognormal distribution with location parameter $\mu = 0.5$, scale parameter $\sigma = 0.6$, and shift $\tau \geq 0$. The shaded region between 0 and σ demonstrates the concentration of approximately 68% of measurements around the mean (analogous to the empirical rule in normal distributions). The distribution exhibits a characteristic right-skew, with a long tail extending toward larger values. This shape naturally captures the behavior of execution times: a minimum unavoidable overhead (τ) followed by a rapid rise in probability density near the center, with diminishing probability for increasingly large outlier measurements.

3.4 Benchmark Results and Comparative Analysis

3.4.1 Contraction on different tensor sizes

The first comparison focuses on the contraction (`ij,jk->ik`) across three tensor sizes.

Results are presented in Figure 3.2.

Before analyzing the results, it is important to define algorithmic overhead: the cost of executing non-computational operations required to perform a computation, such as index calculations, bounds checking, memory address calculations, function call overhead, and temporary buffer allocation. For small problems, the total execution time is dominated by this overhead rather than the actual arithmetic operations (e.g., floating-point multiplications and additions).

Small Problems (10×10): Overhead Dominates

For $(10, 10) \times (10, 10)$ matrices (200 floating-point multiplications total), all implementations complete in sub-microsecond times. At this scale, algorithmic overhead is the main source of latency. **Odometer** exhibits measurable latency even for the trivial 10×10 case, because it must iterate through the combined space and perform dynamic index arithmetic on each step. For a $10 \times 10 \times 10$ iteration space, this means 1,000 coordinate-to-offset calculations. **CodeGen** shows significant improvement because loop bounds are hardcoded, allowing the compiler to eliminate bounds-checking and inline the stride arithmetic as literal constants. The generated C code, when compiled with `-O3`, allows the compiler to unroll loops. **Permute-Flatten** shows overhead from data permutation operations. While the subsequent GEMM is fast, the permutation timings are noticeable for small tensors where computation time itself is negligible. **BLAS** OpenBLAS detects the small problem size and dispatches to a highly-tuned small-matrix kernel with minimal overhead, it's increase in performance in respect to the permute flatten approach is due to how it handles data stored in the cache instead of main memory as described in [1].

In conclusion, at this scale call overhead and memory allocation cost are the main tasks in execution time. CodeGen and BLAS are preferred because they reduce this overhead.

Medium problems (50×50)

At $(50, 50) \times (50, 50)$ (125,000 operations) the working set (3 matrices $\times$ $50^2 \times 8$ bytes = 60 KB) exceeds the per-core L1 cache (32 KB) but remains within L2 cache (256 KB). **Odometer** 125,000 loop iterations accumulate significant overhead. Each iteration requires index coordinate calculations and memory address arithmetic, summing up to a relevant latency. **CodeGen** shows strong improvement over Odometer, with the hardcoded loop structure allowing compiler optimizations. The compiler's ability to unroll the inner loop and maintain data in registers significantly improves temporal locality. **Permute-Flatten** becomes competitive with CodeGen, as the BLAS GEMM operation's efficiency grows with problem size. The overhead of permutation is amortized over computation. **BLAS** remains marginally faster

than Permute-Flatten. This is because BLAS is optimized for cache efficiency, and the 50×50 case is large enough to benefit from its blocking strategies.

In conclusion, the 50×50 dimensions shows how the data access pattern affects performance. CodeGen's compiler-generated code and BLAS's hand-tuned kernels both exploit vectorization and cache locality that our generic iteration strategy cannot match.

Large Problems (100×100): Memory Hierarchy Matters

For $(100, 100) \times (100, 100)$ (1 million operations), the working set (3 matrices $\times$ $100^2 \times 8$ bytes = 240 KB) exceeds the per-core L2 cache and heavily contests the shared L3 cache (16 MB total, shared among 8 cores). **Odometer** execution time becomes prohibitive, as 1 million dynamic index calculations represent 1 million CPU cycles that could be spent on arithmetic. **CodeGen** shows good scaling. The compiler's loop unrolling and prefetcher-friendly access patterns help maintain reasonable performance. However, without explicit cache-blocking (as would require language intrinsics or inline assembly), the generic loop structure still causes memory traffic inefficiency. **BLAS** employs sophisticated cache-blocking strategies to optimize memory utilization during the $100 \times 100 \times 100$ GEMM computation. The algorithm partitions the matrix multiplication into rectangular blocks (e.g., $60 \times 60 \times 320$) that fit entirely within fast L2 cache. **Permute-Flatten** shows slower performance than direct BLAS because the intermediate permutation operations create additional memory traffic and require temporary buffer allocation.

In conclusion, at large scales, the BLAS approach's becomes sensibly better then on smaller datasets while the codegen approach remains competitive. The Odometer approach becomes impractical due to its overhead, while the Permute-Flatten approach suffers from additional memory traffic and temporary buffer allocation.

Higher-Rank Tensors: 3D Contraction

The 3D contraction test (`abc,cd->abd` with shapes $(10, 10, 10) \times (10, 10)$) shows how the implementations handle non-standard tensor patterns. This operation does not map naturally to BLAS Level-2 or Level-3 operations, making it a test of the permute-based reduction strategy.

All implementations produce numerically identical results, but performance varies. **Odometer** handles the operation with no special logic, iterating through the combined space. **CodeGen** generates custom nested loops, specializing entirely to this specific contraction pattern. The loop structure is:

```
for i = 0 to 10
    for j = 0 to 10
        for l = 0 to 10
            for k = 0 to 10
                C[i,j,l] += A[i,j,k] * B[k,l]
```

This structure exhibits good cache locality: the innermost loop over k accesses B contiguously along its second dimension.

Permute-Flatten groups indices to form a (100×10) matrix (the permuted and flattened A) and a (10×10) matrix (B), reducing the operation to GEMM.

The reshaping step introduces overhead, but the subsequent matrix multiplication efficiency compensates for modestly-sized tensors. **BLAS** For higher-rank operations like 3D contractions, BLAS leverages the Permute-Flatten transformation internally, then dispatches the resulting $(100 \times 10) \times (10 \times 10)$ matrix multiplication to its optimized GEMM kernel. The performance advantage comes from BLAS's cache-aware blocking and assembly kernels that increase CPU utilization.

3.4.2 Summary of Implementation Trade-offs

The experimental results validate our implementation hierarchy:

Odometer is a correct, general-purpose reference implementation suitable for small problems and library validation.

CodeGen performs best amongst our implementations but requires a beforehand compilation step which is not accounted for in the benchmark times.

Permute-Flatten provides a useful middle ground for rank-N contractions that don't fit standard BLAS patterns. Its performance depends critically on avoiding unnecessary data copies and permutation overhead.

BLAS performs best on standard operations (GEMM, GEMV, GER, DOT). The performance advantage grows with problem size.

3.5 Higher-Dimensional Tensor Comparisons

Beyond 2D matrix multiplication, we also benchmarked higher-rank tensor operations to assess how implementations scale to more complex contraction patterns. Figure 3.4 shows the performance of all implementations across tensor dimensions 2, 5, and 10 with small problem sizes.

Observations across dimensionality reveal important insights:

The BLAS implementation maintains its performance advantage across all dimensionalities because most higher-rank operations can be decomposed into matrix multiplication via the Permute-Flatten strategy. CodeGen scales well, the Just-in-time specialization shows consistent performance across dimensions. The compiler's ability to optimize custom loop structures is dimension-agnostic. Odometer penalty grows as the overhead of coordinate calculations becomes more pronounced with increasing rank. For rank-5 tensors, the per-iteration cost of computing 5-dimensional offsets accumulates sufficiently to render the approach impractical. Permute-Flatten remains viable as the strategy of reducing arbitrary N-dimensional contractions to matrix multiplication works consistently across all tested dimensionalities, confirming the theoretical foundation presented in Section 2.3.

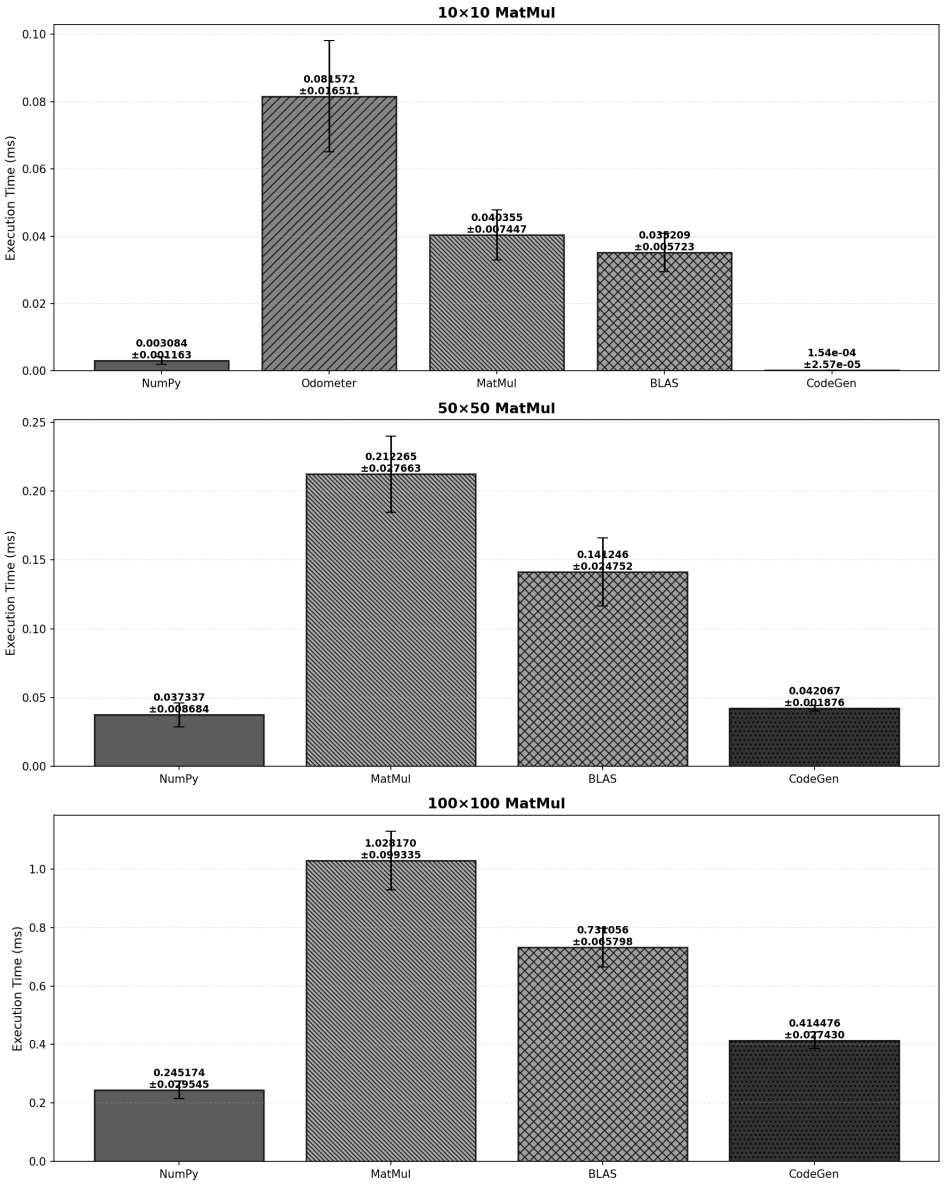


Figure 3.2. Comparative benchmark results across all implementations for matrix multiplication tasks. Bars represent median execution times with 95% confidence intervals derived from lognormal fits. Results demonstrate the BLAS-accelerated and CodeGen approaches achieving superior performance on larger problems.

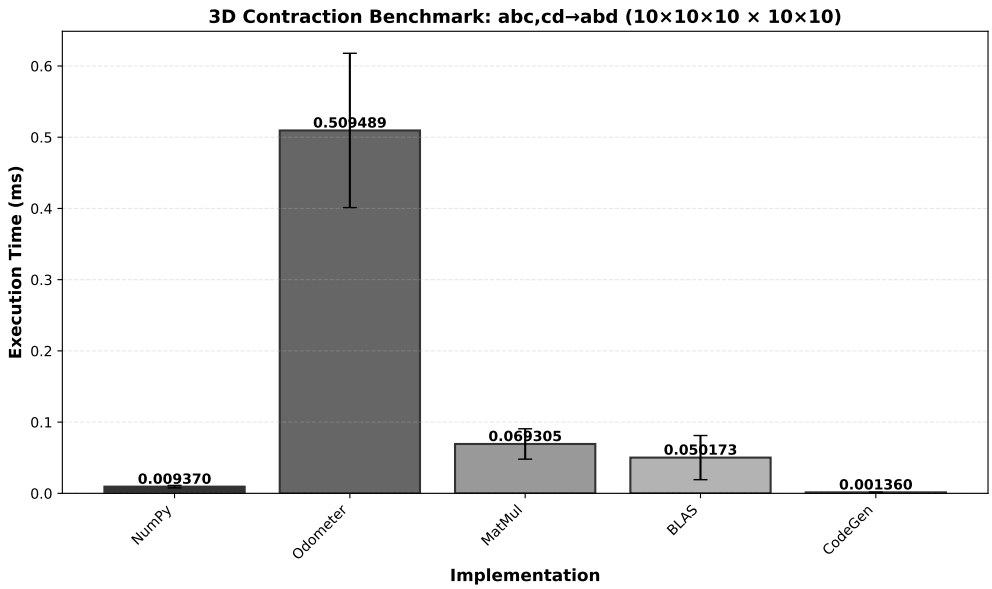


Figure 3.3. Performance comparison for 3D tensor contraction (`abc,cd->abd` with shapes $10 \times 10 \times 10$ and 10×10). Results show CodeGen's superior performance through loop specialization, while BLAS and Permute-Flatten leverage matrix multiplication efficiency. This benchmark demonstrates how all approaches correctly handle non-standard tensor patterns.

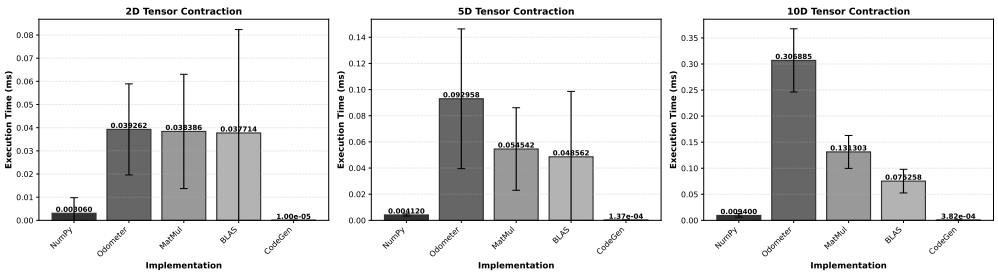


Figure 3.4. Performance comparison across tensor dimensions (2D, 5D, and 10D) with small problem sizes. Test cases use small tensors to isolate algorithmic overhead: 2D tests the fundamental matrix multiplication pattern `"ij,jk->ik"` with 5×5 matrices; 5D and 10D use similarly ranked contraction patterns appropriate to their dimensionality with 3^5 (243 elements) and 2^{10} (1,024 elements) respectively. The results demonstrate how Odometer's overhead grows significantly with dimensionality, while specialized approaches like CodeGen reduce overhead, and BLAS maintains relatively consistent performance across all dimensions. The elevated variance in some configurations—particularly visible for Odometer and CodeGen at higher ranks—reflects the substantial influence of system scheduling noise and cache behavior on small tensor operations, where constant overheads (represented by τ in the lognormal model) constitute a larger proportion of total execution time.

Chapter 4

Conclusions

4.1 Summary

In this work, we have presented a comparison of different implementations for the Einstein summation convention.

The **Odometer** approach serves as a general-purpose interpreter that dynamically evaluates Einstein notation at runtime. While this methodology guarantees correctness and flexibility across arbitrary tensor shapes and contraction patterns, it comes at a significant performance cost. The per-iteration overhead of computing multi-dimensional offsets grows substantially with tensor rank, rendering this approach impractical for high-dimensional operations. Nevertheless, it remains important in this work as a reference implementation for validating the other optimizations.

The **Just-In-Time code generation** by treating Einstein notation as a domain-specific language and generating custom C code for specific tensor shapes, this methods performances benefit from the compiler's optimization capabilities. The primary limitation of this approach is the requirement for a beforehand compilation step, which may introduce latency in scenarios where tensor shapes are determined dynamically.

The **Permute-Flatten** strategy leverages the notion that arbitrary N-dimensional tensor contractions can be reduced to a canonical 2D matrix multiplication through tensor unfolding. Although it incurs overhead from data reorganization and permutation operations, for modestly-sized tensors, this overhead is amortized by the efficiency of the subsequent matrix multiplication.

The **BLAS** The advantage of BLAS grows with problem size due to cache-aware blocking and assembly-level kernel optimizations. However, BLAS performs best on problems that map naturally to its operation set (GEMM, GEMV, GER, DOT). For non-standard contraction patterns, BLAS must fall back to the Permute-Flatten strategy.

From our experimental results we can say that the advantage of specialized approaches over general-purpose interpretation grows with both tensor dimensionality and problem size. For 2D matrix multiplication, all implementations cluster relatively closely in performance, but as dimensionality increases, the overhead of runtime coordinate calculation becomes dominant in the Odometer approach. Similarly,

as tensor dimensions scale upward, the fixed overhead of code generation and permutation operations becomes negligible compared to their impact on larger computational kernels.

4.2 Future works

This research presents different possible improvements. Extension to GPU acceleration represents perhaps the most immediate opportunity, as tensor operations are fundamentally parallelizable and GPUs excel at regular memory access patterns.

Integration of advanced BLAS backends such as Tensor Processing Units and specialized hardware accelerators could yield further performance improvements.

Finally, investigation into how machine learning frameworks might integrate these insights into their runtime systems could yield significant practical benefits. Automatic dispatch between implementations, intelligent memory layout selection, and adaptive compilation strategies all represent possible directions for future works.

Bibliography

[1] Jack Dongarra, Sven Hammarling, Nicholas J. Higham, Samuel D. Relton, Pedro Valero-Lara, and Mawussi Zounon. The design and performance of batched blas on modern high-performance computing systems. *Procedia Computer Science*, 108:495–504, 2017. International Conference on Computational Science, ICCS 2017, 12-14 June 2017, Zurich, Switzerland.

[2] Albert Einstein. The foundation of the general theory of relativity. *Annalen der Physik*, 354(7):769–822, 1916. The original introduction of the summation convention and index contraction.

[3] Morton E. Gurtin. *An Introduction to Continuum Mechanics*. Academic Press, 1981.

[4] Mikhail Itskov. *Tensor Algebra and Tensor Analysis for Engineers*. Springer, 6th edition, 2019.

[5] Julien Klaus, Mark Blacher, and Joachim Giesen. Compiling tensor expressions into einsum. In *International Conference on Conceptual Structures*, 2023.

[6] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009. Standard reference for computational tensor contractions and n-mode products.

[7] David A. Patterson and John L. Hennessy. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2018. Chapter 2 provides comprehensive coverage of memory hierarchies, cache structure, and their performance implications.

[8] Stefan Roberts, Spyros Gidaris, Nikos Komodakis, and Patrick Kyriazis. Einsum is all you need. *arXiv preprint arXiv:2010.13454*, 2020. Discusses the equivalence of einsum operations to matricized GEMM operations.

[9] K. Åhlander. Einstein summation for multidimensional arrays. *Computers & Mathematics with Applications*, 44(8):1007–1017, 2002.